

4×4 Torus Gold Router 设计说明

基于课程资料与当前已确认设计细节整理

设计范围

本次 gold router 面向 4×4 torus 路由网络，是在课程原始 gold router 设计思想基础上的二维扩展版本。它保留了原设计中的关键骨架，包括 64-bit fixed-size packet、2 个 virtual channels、input/output channel buffers、clocked synchronous router、ready/send 握手机制，以及 virtual cut-through 风格的逐跳转发。

拓扑	4×4 torus
物理端口	N / S / W / E / PE
Packet 宽度	64 bit，其中高 32 bit 为 header
VC 数量	每个物理端口 2 个 VC
当前实现	single-slot VC buffer 版本

拓扑与端口

每个 router 具有 5 个物理端口：N、S、W、E、PE。其中 PE 用于与本地 processor element 或 NIC 交互，其余四个方向端口用于连接相邻 router。当前确认的连接方式是同名 side 对接，也就是 N input 连接邻居的 N output，S/W/E 方向同理。因此，一个 packet 若要继续沿某个方向前进，在 router 内部通常表现为“同方向输入到同方向输出”，例如沿 E 方向继续前进时是 E_in → E_out，而不是对穿式转发。

数据包与 Header

当前确认的 packet 为固定 64 bit，其中高 32 bit 为 header，低 32 bit 作为 payload。32-bit header 定义如下：header[31] 为 vc，header[30] 为 dirx，header[29] 为 diry，header[28:25] 为 hops_x，header[24:21] 为 hops_y，header[20:17] 为 source_x，header[16:13] 为 source_y，header[12:9] 为 dest_x，header[8:5] 为 dest_y，header[4:0] 为 reserved。dirx 编码为 0=E、1=W，diry 编码为 0=S、1=N。source_x/source_y 与 dest_x/dest_y 主要用于调试、波形分析和 testbench 检查。

路由算法

当前 router 采用 deterministic minimal routing、source routing 与 dimension-order routing 的组合。路径由 NIC 在 packet 注入前预先计算好，router 本身不重新计算整条路径，而只依据 header 中的剩余跳数和方向逐跳转发。整体顺序固定为先 X 后 Y：当 hops_x 大于 0 时，packet 处于 X phase，根据 dirx 请求 E_out 或 W_out，并在内部 forwarding 成功时将 hops_x 减一；当 hops_x 等于 0 且 hops_y 大于 0 时，packet 进入 Y phase，根据 diry 请求 N_out 或 S_out，并在内部 forwarding 成功时将 hops_y 减一；当 hops_x 和 hops_y 都为 0 时，packet 到达目的节点，请求 PE_out。方向选择采用最短路径规则，其中 X 维若两侧距离相等则固定选 E，Y 维若两侧距离相等则固定选 S。

Virtual Channel 规则

每个物理端口包含 2 个 VC，但在 header 中只保留 1 bit 的 vc 字段。这里的 vc 不是固定历史标签，而是“当前下一跳要使用的 VC bit”。逻辑上的通道类别顺序定义为 $X0 < X1 < Y0 < Y1$ ：在 X phase 中，vc=0/1 分别表示 X0/X1；在 Y phase 中，vc=0/1 分别表示 Y0/Y1。因此，在 X→Y 切换点，vc 可以从 1 改回 0，这表示合法的 X1→Y0 转换，而不是违规回退。这个 1-bit VC 方案成立的关键前提是，VC 选择逻辑不仅依赖 header，也依赖当前 packet 所在的输入端口，因为输入端口身份在 router 结构上天然已知。

Dateline 与 VC 更新

为避免 torus 中由环形依赖引起的死锁，当前设计引入 dateline 规则。在 X 维上，若 dirx=E 且 local_x=3，或者 dirx=W 且 local_x=0，则本跳跨越 X dateline；在 Y 维上，若 diry=S 且 local_y=3，或者 diry=N 且 local_y=0，则本跳跨越 Y dateline。VC 更新规则确认如下：在 X phase 中， $next_vc = vc \mid cross_x_dl$ ；在 Y phase 的第一跳，如果 packet 来自 E_in、W_in 或 PE_in，则 $next_vc = cross_y_dl$ ；在 Y phase 中继续前进时，如果 packet 来自 N_in 或 S_in，则 $next_vc = vc \mid cross_y_dl$ 。这样即可正确支持 X0→X1→Y0→Y1 的类别顺序。

Buffer 组织与内部结构

当前版本采用 single-slot VC buffer，也就是每个物理端口具有 2 个 input VC buffer 和 2 个 output VC buffer，但每个 VC buffer 深度只有 1，不实现更深的 FIFO。这样做的好处是结构清晰、控制简单，也更贴近当前的握手时序和课程中的教学版节奏。逻辑上，router 可拆分为输入控制、路由计算、request 生成、输出控制、仲裁器，以及 input-to-output 的数据通路几部分。

输出口合法请求者与仲裁

当前确认的 5 个输出口合法请求者集合如下：E_out 接受 {E_in, PE_in}；W_out 接受 {W_in, PE_in}；N_out 接受 {N_in, E_in, W_in, PE_in}；S_out 接受 {S_in, E_in, W_in, PE_in}；PE_out 接受 {N_in, S_in, E_in, W_in}。在此基础上，request legality 进一步定义为：E_out 只接受满足 hops_x > 0 且 dirx=E 的 E_in 或 PE_in；W_out 只接受满足 hops_x > 0 且 dirx=W 的 W_in 或 PE_in；N_out 接受满足 hops_x = 0、hops_y > 0 且 diry=N 的 N_in/E_in/W_in/PE_in；S_out 接受满足 hops_x = 0、hops_y > 0 且 diry=S 的 S_in/E_in/W_in/PE_in；PE_out 接受来自四个网络方向且满足 hops_x = 0、hops_y = 0 的 packet。仲裁采用 round-robin / circular rotating priority，并按每个 output port、每个 VC 独立维护优先级状态。当前确认的优先级顺序为：E_out 为 E_in → PE_in；W_out 为 W_in → PE_in；N_out 为 N_in → E_in → W_in → PE_in；S_out 为 S_in → E_in → W_in → PE_in；PE_out 为 N_in → E_in → S_in → W_in。

握手与时序

router-router 之间采用同步采样的数据传输与组合产生的 ready/send 信号。每个物理端口都具有 si/ri/di[63:0] 和 so/ro/do[63:0] 两组接口，并且全网共享 clk、reset 和 polarity。reset 采用 active-high synchronous reset；polarity 在 reset 时为 0，释放后每个周期翻转一次。当前确认采用 internal_vc = polarity、external_vc = ~polarity，也就是一个周期中 router 内部只处理一个 VC，而外部链路只传另一个 VC。ri 采用 Moore 风格，本质上表示当前 external_vc 对应的 input buffer 是否为空；so 采用 Mealy 风格，

本质上表示当前 external_vc 的 output buffer 有效且对端 ready 为 1。一次成功的 router-router 发送不是纯异步电平触发，而是本周期组合逻辑满足发送条件，在该周期结束的时钟上升沿，接收端把 do 写入对应 input buffer，发送端清空对应 output buffer。无阻塞时，一个 packet 每经过一跳需要 2 个周期：1 个周期完成 router 内部 input→output forwarding，1 个周期完成 router→router 的外部传输。header 的更新，包括 hops_x/hops_y 递减以及 vc 更新，都在内部 forwarding 成功、写入 output buffer 的那个时钟沿完成。

当前实现状态

基于以上已确认规格，当前 gold_router 采用的是 single-slot VC buffer 版本，设计重点放在功能正确、路由路径正确、VC 切换规则正确、握手时序正确以及仲裁逻辑正确，而不是先追求更深 buffer 带来的吞吐提升。后续如果需要，可以在不改变路由算法和 header 结构的前提下，把每个 VC buffer 从 1-entry 升级为更深的 FIFO，以进一步降低阻塞概率。

关键确认项速览

输出口	合法请求者集合
E_out	{E_in, PE_in}
W_out	{W_in, PE_in}
N_out	{N_in, E_in, W_in, PE_in}
S_out	{S_in, E_in, W_in, PE_in}
PE_out	{N_in, S_in, E_in, W_in}